Module 6 — Claude Agent SDK

Lesson 6.4

Prompt caching for cost and latency

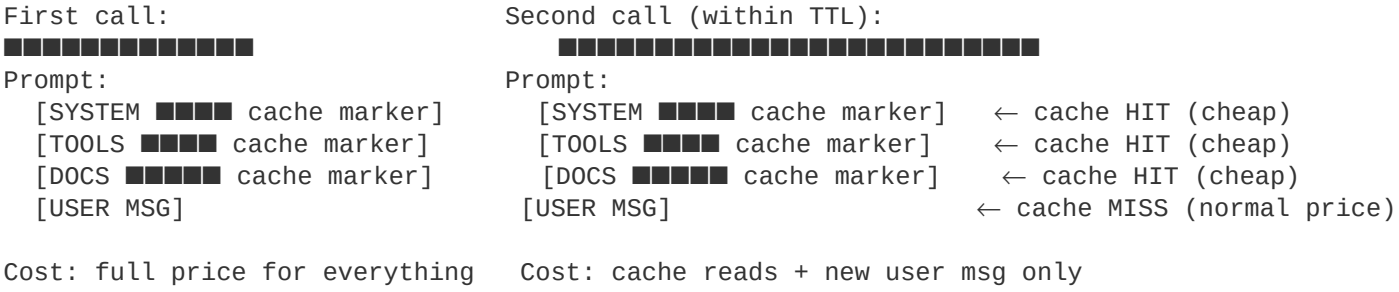
Mark stable parts of your prompt as cacheable. Subsequent calls within the cache TTL pay a fraction of input cost and respond faster.

Why it matters

Agent runs reuse the same system prompt, tool definitions, and often the same large file contents across many model turns. Without caching, each turn re-bills all of that as input tokens. With caching, repeated inputs cost a small fraction (the cache read price), often dropping total spend 70–90% for typical agent workloads.

For long-running agents or high-volume automation, prompt caching is the single biggest cost lever.

How it works (mental model)



Cache markers (`cache_control: {"type": "ephemeral"}`) are placed at strategic points in the prompt. The model finds the longest matching prefix in cache and only computes from the divergence point.

The Anthropic API supports up to ~4 cache markers per request. Place them at boundaries between stable and dynamic content.

How the SDK exposes it

The Claude Agent SDK applies caching automatically for the parts of the prompt it controls (system prompt, tool definitions, conversation history). You typically don't have to mark them yourself.

What you can do:

- **Add stable content to the system prompt** — anything stable goes in the system prompt and gets cached automatically.
- **Avoid varying stable content** — if you change a comma in the system prompt each run, you blow the cache. Keep it byte-stable.
- **Use longer-lived sessions** — caches have a TTL (typically 5 minutes, 1 hour on extended tiers). Runs spaced close together hit the cache; far-apart runs don't.

When you use the raw Anthropic API directly (or build your own multi-call orchestrator), you mark caches explicitly:

```
client.messages.create(
    model="claude-opus-4-7",
    system=[
        {"type": "text", "text": LONG_STABLE_SYSTEM_PROMPT, "cache_control": {"type":
    "ephemeral"}},
    ],
    tools=[
        # ... tools ...
```

```
        # Final tool can have cache_control to cache the whole tools block
    ],
    messages=[
        {"role": "user", "content": [
            {"type": "text", "text": LARGE_STABLE_DOC, "cache_control": {"type": "ephemeral"}},
            {"type": "text", "text": this_turn_user_text},
        ]},
    ],
)
```

The last cache marker in the request anchors caching up to that point.

What to cache

Cache things that are big AND stable:

Cache-worthy	Not worth caching
System prompt (your agent's persona, rules)	A 50-token user question
Tool definitions (often kilobytes)	A timestamp
Reference docs the agent always needs	Small dynamic context
A code style guide, a schema, a brand voice doc	Per-turn rapidly-changing state

The minimum block size that's worth caching depends on the API — generally on the order of 1024 tokens. Don't cache tiny snippets.

TTL considerations

- **Default TTL: 5 minutes.** A turn-by-turn agent run that completes in <5 min will hit cache on every turn after the first.
- **Extended TTL: up to ~1 hour** (request-specific, may require a higher tier). Used for "the user might come back within an hour" scenarios.
- **Cache misses after TTL.** Long pauses between turns blow the cache.

For batch automation that runs every N minutes, caching pays if N < TTL.

Measuring impact

The API returns `usage` info on each response, including cache hits:

```
usage = response.usage
# {
#   "input_tokens": 25,                    # what was newly billed
#   "cache_creation_input_tokens": 10000, # paid to write cache (first time only)
#   "cache_read_input_tokens": 9500,      # cheap cache reads
#   "output_tokens": 1500,
# }
```

A healthy long-running agent shows large `cache_read_input_tokens` and small `input_tokens` on most turns.

Add a small wrapper that logs these per-turn — five lines that pay for themselves.

Common cache-breaking mistakes

1. **Per-run timestamps in the system prompt.** If you template `"Today is {today}"` into the system prompt, every day blows the cache.
2. **Tool definitions that include dynamic state.** Tool descriptions should be stable. Move dynamic state into tool arguments.
3. **Reshuffling tool order.** Cache keys on byte sequence; reordering tools is a different prefix.
4. **Including run-id or trace-id in cached blocks.** Same problem — varies per run.

Audit a healthy agent: print the cache markers' content as a hash on every run. If hashes change, you've drifted.

Try it

For the example SDK agent:

1. Run a task with a long system prompt 5 times in quick succession. Check `usage` per run. After the first, you should see large `cache_read_input_tokens` and small `input_tokens`.
2. Insert `f"timestamp: {time.time()}"` into the system prompt. Re-run 5 times. Note how cache reads drop to zero.
3. Remove the timestamp. Confirm cache reads come back.

Gotchas

- **Cache pricing is non-zero.** Cache reads are cheaper than fresh input but not free. Tiny caches aren't worth the bookkeeping.
- **Caches are per-account, sometimes per-project.** Don't expect cross-account sharing.
- **Cache creation costs more than no-cache** for that first call. Net win only if you'll hit it again within TTL.
- **Streaming and caching interact** — same content, just different transport. No special handling.

Further reading

- Anthropic prompt caching docs: https://docs.claude.com/en/docs/build-with-claude/prompt-caching
- 6.5 Production patterns — observability includes cache metrics
- 6.3 Tool use patterns — stable tool definitions are essential

Next

→ 6.5 Production patterns